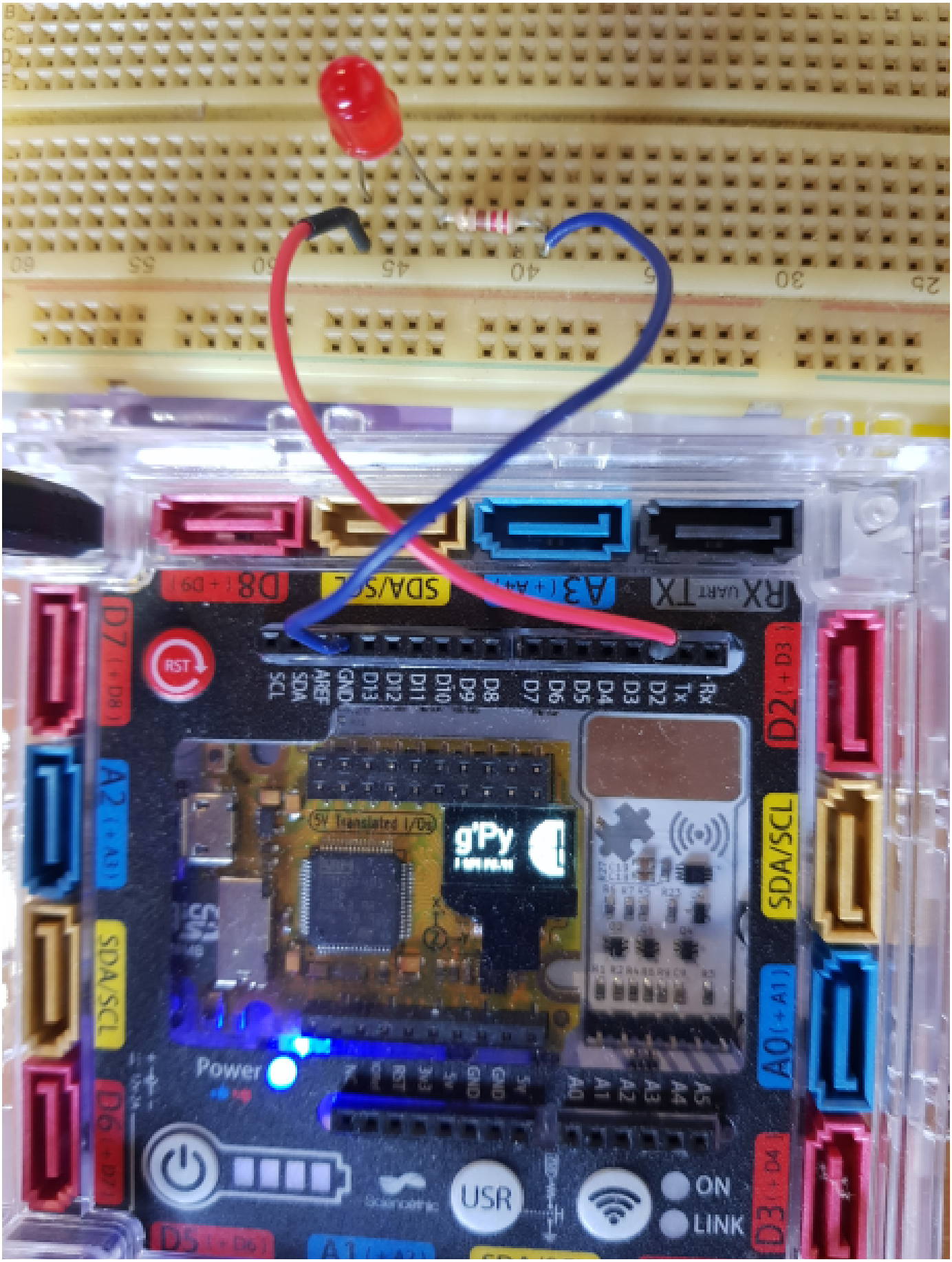
Hello World

Il est coutume de commencer l'apprentissage d'un nouveau langage de programmation par le code d'affichage de la phrase *Hello World*. La déclinaison dans le monde des microcontrôleurs est le clignotement d'une led.

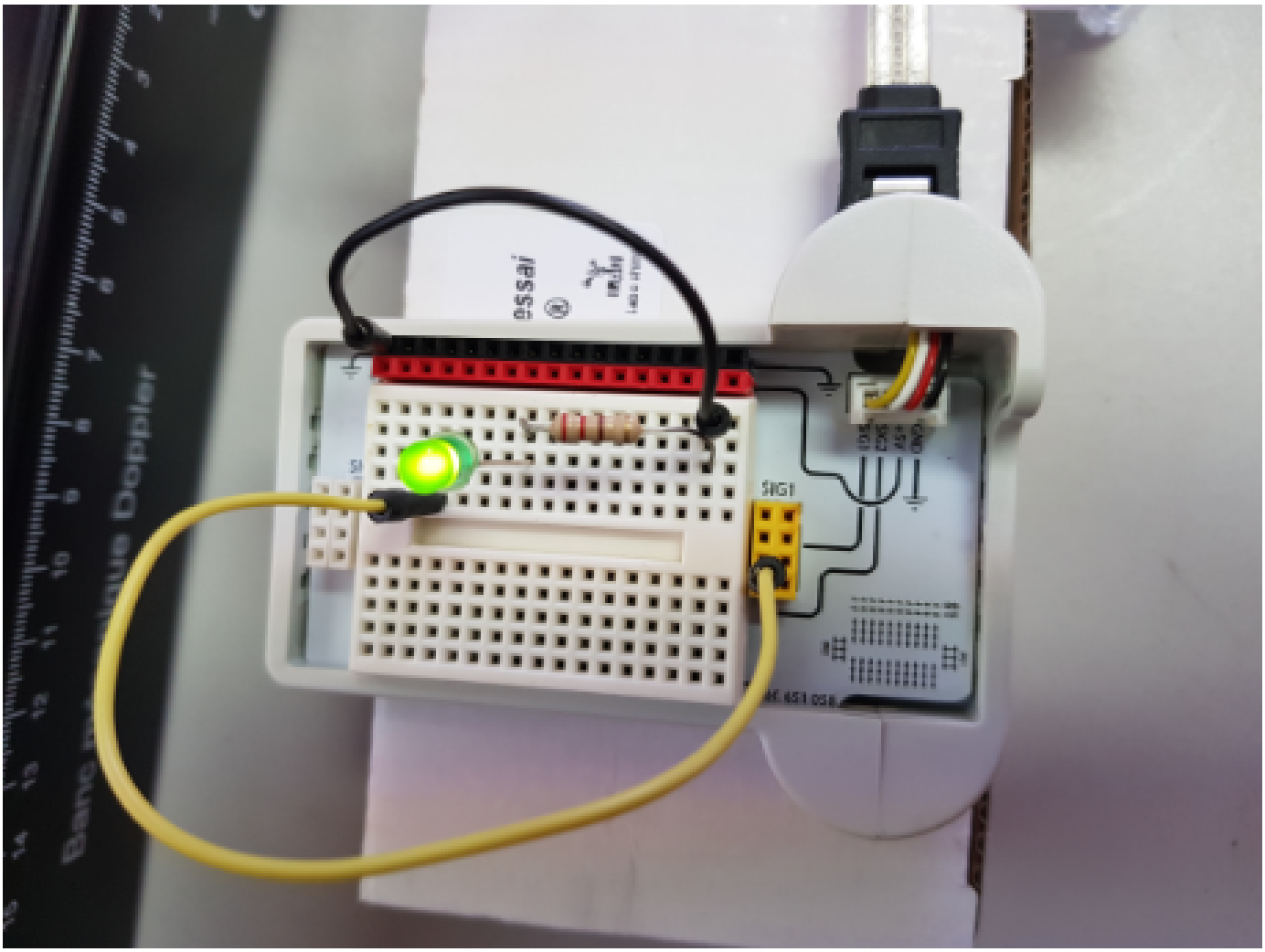
Le montage

On connecte une led sur la broche `D2` du boîtier en série avec une résistance de $220\ \Omega$, en utilisant les connecteurs SIL (*solution 1*) ou SATA associés à la platine de câblage (*solution 2*) de Sciencethic.

Solution 1


D7 (+D8)
A2 (+A3)
SDA/SCL
D6 (+D7)
D5 (+D6)
D8 (+D9)
SDA/SCL
A3 (+A4)
RX UART TX
RST
SCL
AREF
SDA
GND
D13
D12
D11
D10
D9
D8
D7
D6
D5
D4
D3
D2
Tx
Rx
5V Translated I/Os
g'Py
Power
USR
ON
LINK
D2 (+D3)
SDA/SCL
A0 (+A1)
D3 (+D4)

Solution 2



Dialogue via un terminal REPL

Après avoir établi une connexion avec le Plug'Py, on va tester via l'interpréteur (REPL) quelques instructions, en python, destinées à allumer puis éteindre une led.
On peut le faire avec `PuTTY` ou `Thonny`

```
/dev/ttyACM0 - PuTTY

>>> import ppy
>>> dir(ppy)
['__class__', '__name__', 'main', 'stop', 'ADC', 'Accel', 'DAC', 'ExtInt', 'I2C'
, 'Oled', 'Pin', 'RTC', 'SD', 'SDCard', 'SmartLED', 'Switch', 'Timer', 'UART', '
USB_HID', 'USB_VCP', 'bootloader', 'delay', 'disable_irq', 'elapsed_micros', 'el
apsed_millis', 'enable_irq', 'fault_debug', 'freq', 'hard_reset', 'have_cdc', 'h
id', 'hid_keyboard', 'hid_mouse', 'info', 'micros', 'millis', 'mount', 'repl_inf
o', 'repl_uart', 'rng', 'standby', 'sync', 'udelay', 'unique_id', 'usb_mode', 'w
fi']
>>> from ppy import delay, Pin
>>> ma_led = Pin('D2', Pin.OUT)
>>> led.on()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'led' isn't defined
>>> ma_led.on()
>>> delay(500) # provoque une attente de 500 ms
>>>
```

```
Thonny - <untitled> @ 1 : 1
Fichier  Édition  Affichage  Exécuter  Outils  Aide
<untitled>
1

Console
>>> from ppy import delay, Pin
>>> ma_led = Pin('D2', Pin.OUT)
>>> led.on()
  Traceback (most recent call last):
    File "<stdin>", line 1, in <module>
  NameError: name 'led' isn't defined
>>> ma_led.on()
>>> delay(500)
>>> ma_led.off()
>>>
MicroPython (générique)
```

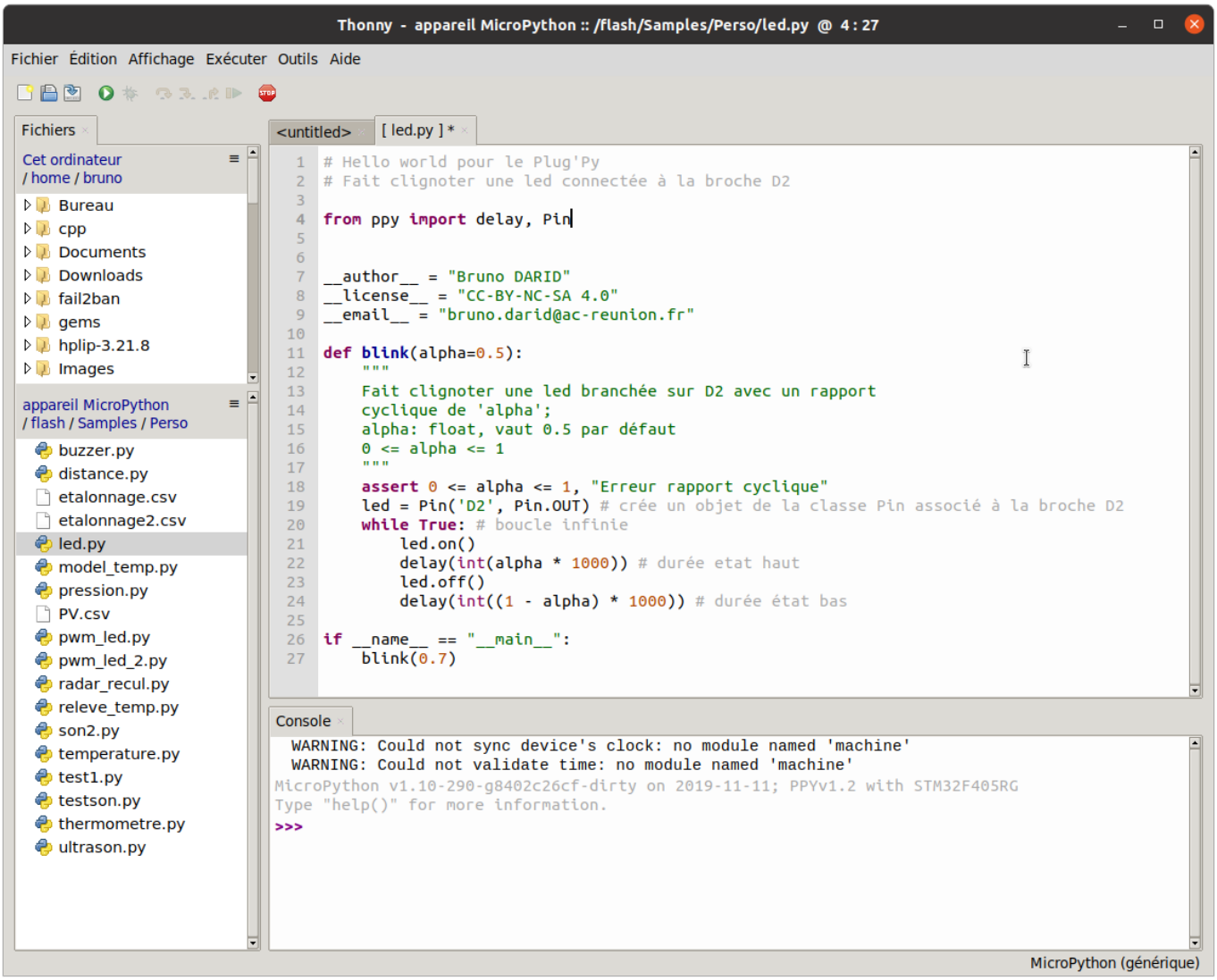
Commentaires du code

L'essentiel des fonctionnalités de la carte réside dans le module `ppy`. Par conséquent, on importera presque systématiquement ce module avec `import ppy` ou uniquement quelques fonctions ou classes par `from ppy import ...`.

Avant de pouvoir utiliser une broche du microcontrôleur en sortie ou en entrée, il faut créer un *objet* de la classe `Pin` qui référence cette broche. La syntaxe est la suivante:

```
ma_led = Pin('D2', Pin.OUT)
```

Les instructions `ma_led.on()` et `ma_led.off()` permettent de placer un niveau haut ou bas de tension sur la broche référencée par `ma_led`. Enfin, `delay(...)` provoque une attente (la valeur en milliseconde doit être placée en paramètre).

Ecriture d'un script

Afin de sauvegarder son travail, on peut naturellement enregistrer son code dans un fichier sur la carte (avec les recommandations déjà faites sur le lieu de stockage).